STŘEDOŠKOLSKÁ ODBORNÁ ČINNOST

Obor č. 18: Informatika

PhishGuard: Aplikace na filtrování podvodných webů

Šimon Semerád
Plzeňský kraj

Plzeň 2025

STŘEDOŠKOLSKÁ ODBORNÁ ČINNOST

Obor č. 18: Informatika

PhishGuard: Aplikace na filtrování podvodných webů

PhishGuard: Application for filtering fraudulent websites

Autor: Šimon Semerád
Škola: Vyšší odborná škola a Střední průmyslová škola elektrotechnická Plzeň, Koterovská 828/85, 326 00 Plzeň
Kraj: Plzeňský kraj
Konzultant: Mgr. Tomáš Nový

Plzeň, 2025

Prohlášení

Prohlašuji, že jsem svou práci SOČ vypracoval/a samostatně a použil/a jsem pouze prameny a literaturu uvedené v seznamu bibliografických záznamů.

Prohlašuji, že tištěná verze a elektronická verze soutěžní práce SOČ jsou shodné.

Nemám závažný důvod proti zpřístupňování této práce v souladu se zákonem č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon) ve znění pozdějších předpisů.

V Plzni dne 11.2.2025 ……

Šimon Semerád

Poděkování

Děkuji panu učiteli Tomáši Novému nejen za neocenitelnou podporu při práci na projektu, ale také za celé čtyři roky inspirativní výuky. Velmi si vážím jeho trpělivosti, odborných rad a vedení nejen při tomto projektu, což pro mě mělo obrovský přínos.

Dále bych chtěl poděkovat Národnímu úřadu pro kybernetickou a informační bezpečnost (NÚ-KIB) za jejich pomoc, ochotu a podporu mého projektu. Jejich odborné rady a vstřícný přístup mě motivovaly a významně přispěly k jeho realizaci.

Anotace

Ve své práci SOČ jsem se zabýval vývojem ochrany proti phishingu. Tento projekt zahrnuje desktopovou aplikaci v Pythonu, která spolupracuje s vlastním rozšířením do prohlížeče pro detekci falešných webových stránek. Cílem bylo vytvořit efektivní nástroj, který dokáže v reálném čase identifikovat phishingové stránky a ochránit tak uživatele před ztrátou citlivých údajů. Výsledkem je funkční řešení s vysokou přesností detekce. PhishGuard byl rovněž navržen s důrazem na uživatelskou přívětivost a snadnou integraci pro každého běžného uživatele. Tato práce přispívá k rozvoji nástrojů pro kybernetickou bezpečnost a osvětě v této oblasti.

Klíčová slova

Phishing; Kyberbezpečnost; Web; Ochrana; PhishGuard

Annotation

In my SOČ project, I focused on developing protection against phishing. This project includes a desktop application in Python that collaborates with a custom browser extension to detect fake websites. The goal was to create an efficient tool capable of identifying phishing sites in real time, protecting users from the loss of sensitive data. The result is a functional solution with high detection accuracy. PhishGuard was also designed with an emphasis on user-friendliness and easy integration for every regular user. This work contributes to the development of tools for cybersecurity and raising awareness in this field.

Keywords

Phishing; Cybersecurity; Web; Protection; PhishGuard

Obsah

1 Úvod 6
2 Analýza 7
3 Technické řešení 8
3.1 Programovací jazyk 9
3.2 Uživatelské rozhraní 9
3.3 Komunikace s webovým prohlížečem 10
3.4 Vlastní globální databáze 12
4 Kontroly 14
4.1 Phishing databáze 14
4.2 Umělá inteligence 16
4.3 Registrace domény 20
4.4 SSL/TLS certifikát 22
4.5 Redirects (přesměrování) 24
4.6 Formuláře 25
5 PhishGuard jako celek 26
6 Závěr 27
7 Použitá literatura 28
8 Seznam obrázků a tabulek 28

1 ÚVOD

Phishing představuje jednu z nejrozšířenějších kybernetických hrozeb současnosti a jeho výskyt každým rokem neustále roste. Jedná se o podvodnou techniku, při které útočníci lákají citlivé údaje od uživatelů prostřednictvím falešných webových stránek, e-mailů nebo zpráv. Podle statistik CSIRT.CZ jsou každoročně evidovány tisíce případů phishingových útoků, avšak skutečný počet obětí je mnohem vyšší. Mnoho incidentů totiž není nahlášeno, ať už z neznalosti, nebo obavy z následků. Útočníci své praktiky neustále zdokonalují, což ztěžuje jejich detekci i pro zkušené uživatele internetu.

Existuje sice celá řada nástrojů zaměřených na boj proti phishingovým útokům, jako je například Google Safe Browsing, tyto nástroje však mají své limity. Podvodné weby často využívají techniky, které jim umožňují tato řešení obejít, nebo jsou detekovány až s výrazným zpožděním. To dává útočníkům dostatek času k tomu, aby napáchali škody, než je stránka odhalena a zablokována. Výzvou tak zůstává vytvoření nástroje, který dokáže rychle a přesně odhalit phishingové stránky bez nutnosti čekat na jejich manuální analýzu či nahlášení.

Dalším problémem je, že mnoho institucí se v oblasti phishingu zaměřuje především na osvětu. Ačkoliv vzdělávání uživatelů o hrozbách a rizicích je nesmírně důležité, samo o sobě nestačí. Starší osoby nebo méně technicky zdatní uživatelé často nemají dostatek zkušeností nebo schopností rozeznat podvodné stránky od těch legitimních. Útočníci navíc stále více využívají sofistikované metody, jako jsou například falešné weby s platnými SSL certifikáty nebo adresy, které na první pohled vypadají důvěryhodně.

Phishing tak představuje významné riziko pro všechny uživatele internetu, bez ohledu na jejich technické znalosti. Jeho vliv sahá od jednotlivců, kteří přicházejí o úspory, až po organizace, které mohou utrpět finanční ztráty či narušení reputace. S ohledem na neustále rostoucí hrozbu a limity stávajících nástrojů jsem se rozhodl vytvořit software, který nabídne efektivní a intuitivní ochranu před phishingem.

Na základě této myšlenky vznikl projekt PhishGuard, jehož cílem je překonat limity stávajících řešení. PhishGuard se zaměřuje na detekci phishingových webů v reálném čase a ochranu uživatelů před ztrátou citlivých údajů. Software byl navržen tak, aby byl snadno použitelný i pro méně zkušené uživatele, a zároveň poskytoval vysokou míru přesnosti v detekci. Věřím, že tento projekt může přispět k ochraně uživatelů a zvýšení bezpečnosti v online prostředí.

2 ANALÝZA

Prvním krokem ve vývoji PhishGuardu byla analýza technik, které phishingové útoky využívají. Bylo nezbytné podrobně pochopit způsoby, jakými útočníci napadají uživatele, jaké jsou jejich cíle a jak dokážou obcházet bezpečnostní opatření, aby bylo možné vytvořit dostatečně účinnou ochranu.

Phishing lze rozdělit do různých kategorií. Nejčastější formou je tzv. **„mass phishing“**, kde si útočníci vybírají za cíl větší skupiny uživatelů a nejčastěji pomocí emailu šíří phishing ve snaze okrást co největší počet lidí. Útočníci zde využívají podvržené vzhledy stránek, zkomolené URL adresy nebo falešné SSL certifikáty, aby oklamali své cíle a získali přihlašovací údaje či jiné citlivé informace. Další pokročilé útoky zahrnují například podvodné investiční kampaně, které se zaměřují na získání finančních prostředků oběti.

Hlavními rysy těchto phishingových webů jsou:

- **Zkomolené URL adresy:** Domény podobné originálům (např. místo facebook.com/login doména facebook_official-login.com).
- **Podezřelé formuláře:** Formuláře na data, které jsou tam, kde by většinou být neměly.
- **Absence nebo nesprávnost certifikace:** Často jsou u phishingu využívány falešné či prošlé SSL/TLS certifikáty.
- **Napodobený design:** Hlavním rysem phishingu je napodobený, avšak nefunkční design napodobující oficiální web.

Tyto rysy sice platí pro většinu podvodných webů, ovšem není to pravidlo, jelikož nemalé procento útočníků našlo způsob, jak většinu těchto podvodných rysů schovat. Tím právě brání v účinném zásahu většině antiphishingových nástrojů, co na trhu jsou.

Jedním z nejrozšířenější nástrojů, co na trhu pro detekci phishingu existuje, je Google Safe Browsing (dále jen GSB). GSB funguje na principu porovnávání hashů URL s databázemi známých škodlivých webů.

V principu:

- GSB používá SHA-256 pro vytvoření hashe url.
- Tyto hashe jsou porovnány s existující databází nebezpečných webů.
- Pokud je nalezena shoda, GSB posílá požadavek na porovnání hashe celkového webu pro odhalení známých vzorů podvodných webových stránek.
- Pokud je i v tomto případě nalezena shoda, uživatel je varován.

I když je tato metoda u většiny kampaní účinná, má několik zásadních problémů:

- **Opoždění v aktualizaci databáze:** Než je nová stránka odhalena, může to trvat několik hodin, ne-li dnů.

- **Nepřímý phishing:** Například podvodné investiční kampaně, které nesbírají nic jiného než kontaktní údaje, nejsou samy o sobě technicky považovány za nebezpečné, tudíž GSB takovéto stránky nedetekuje.
- **Analýza vzorů:** GSB sice analyzuje obsah pomocí známých vzorů, avšak hodně podvodů tyto vzory nesplňuje, tudíž GSB by je neodhalilo.
- **Zaměření na globální kampaně:** GSB často odhalí pouze globálně rozšířené kampaně, ne však lokální kampaně zaměřené pouze na menší skupinu uživatelů.

Na základě této provedené analýzy phishingových praktik a omezení stávajících nástrojů začalo navrhování architektury ochranných mechanismů, které by PhishGuard měl obsahovat. Pro ověření teoretie jsem kontaktoval Národní úřad pro kybernetickou a informační bezpečnost (NÚKIB), který přinesl další cenné poznatky, které značně ovlivnily návrh a implementaci PhishGuardu. Pomocí všech těchto informací byla navrhnuta struktura kontrol PhishGuard:

- **Phishing databáze:** I když tato kontrola není účinná ve všech případech, využití např. GSB či jiných databází nebezpečných webů by se stále nemělo opomíjet.
- **Umělá inteligence:** Využití umělé inteligence se dále rozšiřuje skoro ve všech oblastech informatiky, proto se hodí jí využít i tady, a to zejména funkci Gpt 4, která umožňuje vyhledávání na internetu.
- **Registrace domény:** Z registrace domény pomocí WHOIS serverů se taktéž dá odhalit mnohé podezřelé chování domény.
- **Certifikát:** Stejně jako u domény, z SSL/TTL certifikátů se dá odhalit velké množství podezřelých informací.
- **Přesměrování:** Zjištění toho, jak často a kam se web před otevřením přesměrovává, je další metoda, jak phishing účinně odhalit.
- **Formuláře:** A poslední metodou je kontrola chování formulářů obsažených na webu.

Všechny tyto kontroly dokážou účinně s velkou úspěšností phishing odhalit, podrobnější vysvětlení jednotlivých funkcí je obsaženo dále v dokumentaci. S těmito poznatky bylo zahájeno plánování a vývoj PhishGuardu jako komplexního řešení, které řeší problémy současných nástrojů a zajišťuje vyšší úroveň ochrany běžných uživatelů.

3 TECHNICKÉ ŘEŠENÍ

Nejdůležitější ze začátku vývoje bylo zvolit správné nástroje, formu aplikace, a celkově technický aspekt softwaru.

3.1 Programovací jazyk

První položkou výběru byl programovací jazyk, ve kterém bude PhishGuard fungovat. Zvažovány byly jazyky Python, Java a C++ jakožto jazyky vhodné na tvorbu desktopové aplikace pro Windows, což je přesně forma aplikace, kterou je i PhishGuard.

Nakonec volba padla na Python, jelikož je jednoduchý, přehledný a nabízí velkou škálu knihoven, které usnadní veškeré aspekty vývoje. Taktéž je ze zvažovaných jazyků dle mého názoru nejjednodušší a nejvíc intuitivní. Slabší stránkou Pythonu je tvorba grafického uživatelského rozhraní, ale existuje mnoho knihoven, které umožní tvorbu GUI i v Pythonu.

3.2 Uživatelské rozhraní

Při výběru knihovny bylo zvažováno více variant: Tkinter, PyQT, Kivy a wxPython. Rozhodnutí padlo na Tkinter, a to z několika důvodů.

Je součástí základních knihoven Pythonu, tudíž nevyžaduje žádnou složitou instalaci; je snadno pochopitelná, takže to nijak nezdržuje vývoj; a zároveň pro potřeby PhishGuardu nebylo potřeba tvořit žádné složité GUI, tudíž zvolit jednoduchou a rychlou variantu byla jasná volba. Pomocí Tkinteru bylo vytvořeno jednoduché UI obsahující základní informace o stavu jednotlivých kontrol, které zároveň po dokončení kontrol obsahuje informaci o úrovni důvěryhodnosti dané webové stránky.

Obr. 1: Prvotní uživatelské rozhraní

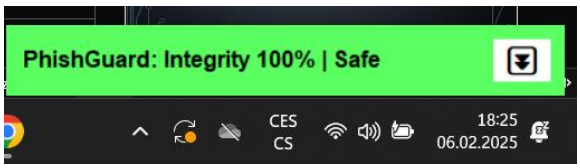
Pomocí knihovny Tkinter taktéž vzniklo notifikační vyskakovací okno, sloužící k upozornění uživatele o výsledku kontroly dané webové stránky.

Notifikační okno funguje na principu parent – child, a je jako child element napojené na hlavní PhishGuard okno. Notifikační okno se inicializuje do globální „Notification“ proměnné. Dále obsahuje metody pro změnu vzhledu podle úrovně bezpečnosti: na zelenou, což značí bezpečný web, nebo na žlutou, což značí podezřelý web, ale ne přímo nebezpečný, a poslední variantou

je červená, což značí přímo nebezpečnou webovou stránku. S využitím přístupu globální proměnné se pak tyto metody v rámci kontrol volají společně s argumentem „Integrity“, což má za výsledek to, že se ve vyskakovacím okně uživateli zobrazí správná barva podle úrovně a zároveň přímo % důvěryhodnosti webové stránky.

Notifikační okno je ve výchozím stavu minimalizované do malé ikonky obsahující logo PhishGuard, tato ikonka se nachází v pravém dolním rohu obrazovky a je nastavená jakožto „overlay“, což znamená, že se zobrazí nad všemi aktivními okny. Při dokončení testu webové stránky okno zůstává minimalizované, pokud je výsledek hodnocení stránky na úrovni „zelené“ nebo „žluté“. V případě „červené“ úrovně se však okno automaticky rozšíří (expanduje), aby uživatel jasně před sebou viděl varování o tom, že se nachází na nebezpečné webové stránce. Uživatel má taktéž možnost sám kdykoliv notifikační okno ručně rozšířit kliknutím na ikonu, čímž zobrazí přesnou informaci o % důvěryhodnosti stránky, a stejným způsobem může okno zpátky minimalizovat, podle potřeb uživatele. Do budoucna je v plánu vytvořit formu aktivní ochrany, která by u nebezpečných webových stránek uživateli přímo zabránila v přístupu.

Obr. 2: Minimalizované notifikační okno



Obr. 3: Expandované notifikační okno

3.3 Komunikace s webovým prohlížečem

Aby bylo možné zajistit předávání URL, které uživatel otevře, v živém čase mezi prohlížečem a desktop aplikací, muselo se vytvořit řešení, které tuhle komunikaci mezi prohlížečem a PhishGuardem umožní.

Po mnohých pokusech jsem se nakonec rozhodl pro využití vlastního rozšíření v prohlížeči zároveň s lokálním serverem pro vyřešení této komunikace.

Rozšíření do prohlížeče momentálně podporuje prohlížeč Chrome (do budoucna se plánuje rozšíření i na další). Je vytvořené v jazyku JavaScript, manifest pro toto rozšíření je napsán ve formátu json, jak je běžné.

Manifest je základní soubor prohlížečového rozšíření, který určuje, jak rozšíření funguje a jaké oprávnění má, ale zároveň určuje i základní informace, jako je název, popis a také verze oprávnění.

Rozšíření pro PhishGuard v již zmíněném manifestu nese název „PhishGuard - URL Monitor“ a vlastní následující oprávnění: tabs, http://localhost:5000 a activetab.

Jak oprávnění „tabs“, tak oprávnění „activetab“ slouží k odchytávání URL, které uživatel otevře. Samotné „tabs“ slouží k tomu, aby odchytil záložku, kterou uživatel nově otevře, zatímco „activetab“ odchytí záložku, na kterou uživatel přepne. Ve výsledku tak PhishGuard odchytí opravdu každý web, který uživatel buďto nově otevře, nebo se na něj vrátí.

Jak jsem již psal na začátku, samotný background script rozšíření je napsán v jazyce javascript, což je ideální pro práci s API prohlížeče. Background script běží neustále na pozadí a vykonává vše, co má, s ohledem na to, jaké má v manifest.json zmíněné oprávnění.

Script se zaměřuje na sledování tří hlavních událostí, a to: aktivaci nové záložky, přepnutí na jinou záložku a změnu obsahu záložky (např. načtení nové stránky). Při vzniku kterékoliv z těchto událostí skript detekuje URL aktuální webové stránky, kterou následně zpracuje a odešle jako HTTP požadavek na lokální Flask server. Pro tyto účely se používají funkce „chrome.tabs.onActivated“ a „chrome.tabs.onUpdated“.

První zmíněná „chrome.tabs.onActivated“ detekuje, pokud uživatel aktivuje (tj. přepne na) jinou záložku v prohlížeči. Samotná funkce je tzv. „Listener“, což znamená,že nepřetržitě naslouchá, a v tomto případě se spustí pokaždé, když uživatel přepne na jinou záložku. Při zjištění této události funkce zjistí ID aktivní záložky. S použitím tohoto ID získá rozšíření veškeré informace o záložce, včetně URL adresy webu, který je v ní otevřen. Pokud URL existuje (např. uživatel nemá otevřené v prohlížeči pdf, obrázek atd.), rozšíření toto URL ve formátu json za pomocí HTTP požadavku (POST) pošle na server.

Druhá z funkcí „chrome.tabs.onUpdated“ naopak detekuje, jestli se obsah záložky změní (např. uživatel načte novou stránku). Stejně jako první funkce i tato je funkcí naslouchací, v tomto případě se ale spustí jen tehdy, když uživatel pouze záložku jakkoliv aktualizuje. Pokud se funkce spustí, nejprve zkontroluje, jestli je aktualizace dokončená, neboli jestli se již stránka plně načetla. Následně stejně jako v předešlém případě získá ID záložky, ze kterého získá URL stránky otevřené v záložce, a po kontrole, že URL skutečně existuje, pošle rozšíření URL na server ve formátu json.

```javascript
chrome.tabs.onActivated.addListener(activeInfo => {
  chrome.tabs.get(activeInfo.tabId, tab => {
    if (tab.url) {
      fetch('http://localhost:5000/new_url', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({ url: tab.url })
      })
      .then(response => response.json())
      .then(data => console.log('Success:', data))
      .catch(error => console.error('Error:', error));
    }
  });
});
```

```javascript
chrome.tabs.onUpdated.addListener((tabId, changeInfo, tab) => {
  if (changeInfo.status === 'complete' && tab.url) {
    fetch('http://localhost:5000/new_url', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ url: tab.url })
    })
    .then(response => response.json())
    .then(data => console.log('Success:', data))
    .catch(error => console.error('Error:', error));
  }
});
```

Obr. 4: Background script pro rozšíření do prohlížeče

Naposledy aplikace pro správnou komunikaci s prohlížečem využívá server, který běží přímo v aplikaci na localhostu uživatele. Tento server je postavený na webovém frameworku s názvem „Flask“, což je framework napsaný v Pythonu a umožňuje snadné vytváření serverového

prostředí pro specifické potřeby. Flask je tzv. „microframework“, což v praxi znamená, že poskytuje jen základní funkcionalitu pro webový server, a další potřebné funkce si může vývojář dodatečně sám doinstalovat. Má jednoduchou architekturu a není výkonnostně náročný, což je pro PhishGuard ideální, jelikož je potřeba jen základní funkcionalita, a díky své jednoduchosti Flask nezpomaluje celkový chod aplikace.

Flask je ze základu nastavený jako „webový server“ neboli naslouchá všem HTTP požadavkům, a obsluhuje všechny základní požadavky, jako je např. GET nebo POST. Každý HTTP požadavek je spojený s určitým handlerem (funkcí), který je definován pomocí dekorátoru (speciální funkce, která umožňuje změnit nebo rozšířit chování jiné funkce či metody bez nutnosti měnit její kód). Tyto handlery pak provádějí logiku zpracování požadavku a vrací odpověď (většinou HTML nebo JSON a statusový kód).

Ze začátku musí být Flask inicializován, aby následně bylo v kódu možné dál pracovat například s konfigurací serveru. V našem případě jde hlavně o to, abychom mohli pracovat s zmíněným dekorátorem pro cesty a utvoření logiky jednotlivých požadavků. Flask je inicializován jako app = Flask(__name__), argument v konstruktoru Flasku slouží k určení cesty k souborům a statickým zdrojům, proto využívám v tomto konkrétním případě formu argumentu „__name__“, což flasku řekne, že má využít aktuální modul (neboli soubor .py, ve kterém se spouští).

Dále je ve PhishGuardu zaveden již zmíněný dekorátor, který z prohlížeče odchytává POST požadavky, ve kterých se nachází URL. Tento dekorátor vypadá následovně: @app.route(‘/new_url’, methods=['POST’]) a pod ním je definována funkce „new_url()“, kde už je samotná logika pro zpracování URL.

Tento dekorátor při přijetí POST požadavku na localhost na cestě „/new_url“ spustí funkci new_url. V praxi to znamená, že pokaždé, když prohlížeč odchytí webovou stránku a pošle ji Flask serveru, spustí se celá logika kontrol PhishGuard. Kontroly tím pádem probíhají okamžitě v reálném čase. Odchycený požadavek, neboli json soubor, je pak zpracován tak, že nejdřív je uložen do proměnné data „data=request.json“ a následně se z něj vypíše do proměné website jen to, co potřebujeme, neboli url adresa, a to následujícím způsobem: website = data.get(‘url’).

V souvislosti s Flask serverem najdeme ještě funkci „run_flask()“, která pouze obsahuje příkaz app.run(port=5000), což spouští přesně náš Flask server na portu 5000. Tahle funkce se následně spouští ve vlastní threadu (vláknu), aby server mohl bežet nezávisle na GUI a dalších funkcích obsažených v PhishGuardu.

3.4 Vlastní globální databáze

Pro zajištění plynulého a efektivního fungování aplikace byl vyvinut systém globální databáze. Tato databáze eliminuje duplicitní kontroly a obecně tak zrychluje veškeré procesy aplikace PhishGuard. Globální databáze vytvořená a obsluhovaná pomocí SQL a hostovaná v systému filess.io slouží jako centrální uložiště, ve kterém jsou uchovány veškeré informace o již

analyzovaných webových stránkách. Tento systém slouží jako prevence opakovaných kontrol, zrychluje tak celý systém PhishGuard a weby, které již jednou byly zkontrolovány, jsou poté zanalyzovány v podstatě ihned. Databáze obsahuje tabulku „phish_data“, která je strukturovaná na ukládání veškerých informací o kontrolách. Její struktura vypadá následovně:

- **URL adresa:** První sloupec je URL adresa webu, která slouží jako ID v naší databázi, a právě podle této informace se pak zpětně z databáze vytahují potřebná data.

- **Integrity:** Nejdůležitější částí databáze a celého systému PhishGuard je systém integrity. Jde o procentuální systém popsaný v kapitole 3.2, který přesně určí, jak moc je web bezpečný či nebezpečný, a tato informace se pak uchovává v tomto sloupečku naší databáze.

- **AI, Doména, SSL, Redirect, Formuláře:** Zbylé sloupečky databáze obsahují informaci o tom, jak proběhly veškeré kontroly. Pokud proběhly bez problému, tento sloupeček obsahuje 1; pokud systém vyhodnotil kontrolu jako negativní, sloupeček obsahuje 0; ještě se používá hodnota 2, pokud kontrola není vyhodnocena přímo negativně, ale ani na 100 % pozitivně. Tyto sloupečky jsou prakticky pouze pro to, aby se následně při zpětném získání dat správně zobrazila vizuální informace, jelikož pro správnost bezpečnosti webů by bohatě postačila informace o úrovni integrity.

Při otevření nové URL aplikace nejdřív ověří, jestli již adresa existuje v databázi. V případě, že je URL nalezena, jsou načteny výsledky analýzy uložené v databázi a aplikace je zobrazí uživateli. Tím se výrazně zmenší čas celkové kontroly a sníží se zátěž na všechny prostředky obsažené v PhishGuard, jako je např. Server WHOIS, AI či databáze. V případě, že URL nalezena není, kontroly probíhají klasicky tak, jak by měly, a všechny informace se v průběhu kontrol zapisují do databáze. Pokud se kontrola přeruší v průběhu (uživatel přepne na jinou URL před dokončením kontroly), jsou veškeré informace k dané URL z databáze smazány, abychom předešli tomu, že by databáze obsahovala neúplné informace o kontrole.

4 KONTROLY

Hlavní a nejdůležitější součást celého systému PhishGuard jsou samozřejmě přímo kontroly, které zajištují samotné určení, zda se jedná o nebezpečnou webovou stránku, či o stránku plně legitimní. Tyto kontroly byly vypracovány zkoumáním desítek vzorků phishingu, ať už z internetových kampaní, kampaní přes email, ale také na základě konzultací s Národním úřadem pro kybernetickou a informační bezpečnost. Kontroly fungují tak, že ze začátku má PhishGuard nastaven systém „integrity“ neboli důvěryhodnosti webu na 100 %, a pokud jednotlivé kontroly neprojdou, až poté se procenta odčítají dle závažnosti. Kontroly se spouští postupně jedna za druhou v pořadí: Phishing databáze, Umělá inteligence, Registrace domény, Certifikát SSL/TLS, Redirects a nakonec formuláře. Tento systém umožní přesně určit každý aspekt webu a co nejlépe odhadnout, zda je legitimní či nikoliv.

4.1 Phishing databáze

Metoda kontroly globální databáze phishingových webových stránek sice není plně účinná, ale rozhodně se vyplatí ji zařadit jako jednu z kontrol. PhishGuard využívá konkrétně největší databázi, která celosvětově existuje, a to Google Safe Browsing. V první fázi se přes API request, uskutečněný knihovnou requests přes metodu POST, pošle požadavek na Google Safe Browsing obsahující právě kontrolovanou webovou stránku pod položkou URL v hlavičce „threatEntries“, v dalších hlavičkách tohoto požadavku pak jsou informace o tzv. „threatTypes“, „platformTypes“ a nakonec „threatEntryType“.

První zmíněná „threatTypes“ obsahuje informaci o tom, jaký typ hrozby v databázi hledáme. V případě PhishGuard jsou v tomto typu hlavičky dvě informace, a to „MALWARE“ a „SOCIAL_ENGINEERING“, což zahrne všechny webové stránky zapsané do databáze přímo za tím účelem, že obsahují nějaký škodlivý malware, ale i podvodné phishingové stránky.

Hlavička „platformTypes“ udává typ platformy, na který má podezřelý web cílit. Může to být platforma jako např. IOS, Windows, Linux a další. V případě PhishGuardu tato hlavička obsahuje položku „ANY_PLATFORM“, což znamená, že hledáme nezávisle na tom, na jakou platformu je podezřelý web hlavně směřován.

Poslední hlavičkou v GSB API request je „threatEntryType“, což jenom dává requestu informaci o tom, jaký formát vstupu do API má očekávat. V našem případě je to „URL“, ale může to být i například „EXECUTABLE“, což by značilo, že jako vstup udáváme binární soubor, či například „IP-RANGE“, což by zase naopak kontrolovalo IP adresu, která by mohla být spojená s jakoukoliv podezřelou aktivitou, a tak dále.

```
api_klic = "AIzaSyD5BPtLzJHFky28SSaDeESzZolllk23cZE"
safe_browsing_url = "https://safebrowsing.googleapis.com/v4/threatMatches:find?key={api_klic} HTTP/1.1"
headers = {
    'Content-Type': 'application/json'
}
payload = {
    "client": {
        "clientId": "PhishGuard",
        "clientVersion": "1.5.2"
    },
    "threatInfo": {
        "threatTypes": ["MALWARE", "SOCIAL_ENGINEERING"],
        "platformTypes": ["ANY_PLATFORM"],
        "threatEntryTypes": ["URL"],
        "threatEntries": [
            {"url": url}
        ]
    }
}
params = {
    'key': api_klic
}
```

Obr. 5: Hlavičky a payload API requestu pro GSB

Po sestavení requestu se správnou URL pak aplikace pouze zpracuje odpověď Google Safe Browsing API. Pokud GSB nalezne shodu, automaticky se důvěryhodnost webové stránky nastavuje na 0 bez ohledu na jakékoliv další kontroly.

Tento přístup je zvolen z toho důvodu, že Google Safe Browsing garantuje, že se na 100 % jedná o phishing či jinak nebezpečný web, tudíž je zbytečné zohledňovat zbylé kontroly.

Nakonec se informace o tom, zda kontrola proběhla v pořádku či nikoliv, zapíše do databáze, aby se předešlo dvojímu kontrolování té samé webové stránky.

4.2 Umělá inteligence

Jedna z nejdůležitějších kontrol je integrace s umělou inteligencí. Chatbot PhishGuard byl vytvořen na platformě „ChatBotKit“ a využívá jazykový model „Gpt-4o-mini“. Byl navržen tak, aby při předložení staženého HTML celé stránky byl schopen určit, zda se jedná o phishingovou stránku či stránku bezpečnou. Po tomto vyhodnocení AI webovou stránku ohodnotí od 1 do 10, přičemž 10 je, pokud je stránka nebezpečná, a následně podle toho se odvíjí odečítání úrovně integrity. Jedná se sice o opravdu „jednoduché“ řešení, ale za to velmi revoluční a účinné. Z výsledků testování vyplývá, že právě AI má ze všech kontrol největší úspěšnost ve správném odhalení podvodné stránky.

Kromě samotného AI, v souvislosti s touto kontrolou PhishGuard obsahuje další 2 metody a týmiž jsou „fetch_page_content“ a „extract_relevant_html“. První zmíněná pomocí knihovny requests stahuje HTML celé stránky které následně předává dál do druhé zmíněné metody, která vytáhne z HTML jen potřebné informace aby se zamezilo zbytečnému přehlcování AI.

AI je ponechána ve svém plain stavu, což znamená, že není nijak speciálně trénovaná, avšak při odhalení phishingu se pomocí této stránky sama dotrénovává. Jsou využívány jenom základní funkce, tudíž není potřeba nijak zásadně měnit chování základního modelu Gpt-4o. Funkce AI je pouze upravená pomocí tzv. „backstory“, což je textový popis, který AI říká, co přesně má dělat a hlavně v jakém formátu výslednou informaci předávat zpět. Backstory základně určuje jaké společné rysy mají tyto phishingové stránky, avšak AI si je pak sama také určuje tyto rysy ať už na základě dat které má již v sobě, či dat které se sama naučila právě z odhalených webových stránek.

AI běží na platformě ChatBotKit a aplikace PhishGuard následně s AI komunikuje přes API. Oproti Google Safe Browsing je komunikace přes API s AI o něco složitější a je potřeba zpracovávat například odpovědi ve formátu json, proto se místo knihovny requests v tomto případě pro komunikaci s AI využívá knihovna curl, která funguje na stejném principu, jenom je pro účely konkrétně AI pohodlnější.

Stejně jako u requests a u jakékoliv jiné knihovny, API funguje v podstatě jako HTTP request, což znamená, že musí obsahovat URL, na kterou request posíláme, hlavičky a pak samotná data, která se v tomto requestu odesílají.

Ovšem u knihovny curl musíme jako první vytvořit objekt curl, což se definuje pomocí pycurl.curl(), který se přiřadí do jakékoliv proměnné, čímž se vytvoří instance curl. Request musí obsahovat URL, na kterou odkazuje, ale samozřejmě taky HTTP metodu, kterou budeme používat. Obojí se v knihovně curl nastavuje pomocí funkce setopt(), v případě URL to je setopt(c.URL, <adresa>), což bude adresa ChatBotKit API s endpointem create sloužící pro vytvoření konverzace, a v případě HTTP metody to je settopt(c.POST,1), což nastaví request na to, aby používal metodu POST, což je přesně náš případ. Přes metodu setopt() se nadále ještě nastaví proměnná tzv. „Buffer“, v níž se uloží data odpovědi API requestu.

V datové části PhishGuard API requestu (která je ve formátu json) pro AI nalezneme spoustu informací, jako např. o jaký typ requestu se jedná, popis requestu atd., což jsou informace pro technické fungování nepodstatné; avšak datová část obsahuje i pro fungování podstatná data, jako např. „type“, „text“, „botId“ nebo „contactId“, všechny tyto json identifikátory udávají důležité informace pro AI, aby náš požadavek správně zpracovala.

Informace „type“ udává, jaký typ zprávy umělé inteligenci posíláme. V případě PhishGuardu se jedná o typ „user“, neboli o klasickou zprávu, jakou by jí poslal běžný uživatel. Tato informace se v API udává, protože můžeme přes API například i měnit Backstory umělé inteligence či jiná nastavení.

Položka „text“ jsou klasická data, která zasíláme v textu zprávy, v našem případě je to html obsah webové stránky kterou chceme od AI zkontrolovat.

Části „botId“ a „contactId“ jsou v obou případech specifické informace platformy ChatBotKit. BotID udává ID chatbota, se kterým chceme konkrétně komunikovat, jelikož na jeden API klíč můžeme mít vytvořených nespočet chatbotů. ContactID zase udává ID „kontaktu“, který má chatbot využít. Kontakt je v podstatě soubor informací, jako např. jméno uživatele a specifické požadavky, jaké má chatbot při komunikaci s tímto uživatelem zohledňovat, další z funkcí kontaktů je též zpětné dohledání konverzací specifického uživatele. V případě PhishGuard je využíván defaultní kontakt, který je již ze základu na platformě ChatBotKit, jelikož v tomhle ohledu nepotřebujeme přes API komunikovat s jakýmikoliv dalšími speciálními požadavky.

V hlavičkách requestu pro API se nenachází nic zvláštního, pouze náš API klíč, aby celá komunikace fungovala, a následně hlavičky „accept“ a „Content-type“, které pouze upřesňují informaci o tom, že odesíláme, a i očekáváme zpět data ve formátu json.

```
data = {
    'name': 'Request',
    'description': 'PhishGuardAppRequest',
    'meta': {
        "additionalProp1": {}
    },
    "contactId": "cly4sddh3064diko6gg5cblom",
    "messages": [
    {
    "type": "user",
    "text": webovka
    }
]
"botId": "cly2xmbhi19g0h1u6lqc38r4t"
}

postfields = json.dumps(data)

# Headers
headers = [
    'Authorization: Bearer sk-f5ccbfae299b07ed30c5b31cb7c82273060e33af78623b0abbd8de64f8a88f06',
    'accept: application/json',
    'Content-Type: application/json'
]
```

Obr. 6: Hlavičky a datová část prvního API requestu pro AI

Po nastavení všech informací a sestavení požadavku se pomocí funkce perform() daný požadavek přes knihovnu curl vykoná a následně uzavřeme přes close() instanci curl, aby zbytečně spuštěná instance nepřetěžovala celou aplikaci.

Přes metodu buffer.getvalue().decode('utf8') načteme data odpovědi umělé inteligence a převedeme je do čitelného formátu, neboli do kódování utf8. Následně tato data ještě zvlášť načteme jako json přes funkci json.loads(), abychom s nimi mohli dále bez problémů pracovat.

Tato přijatá data zatím ještě neobsahují přímo odpověď umělé inteligence, nýbrž informace o právě vytvořené konverzaci. Z dat, která jsme momentálně přijali, si PhishGuard pouze vytahuje položku „key" neboli ID konverzace, přes které se pak dostaneme přímo k odpovědi AI.

Abychom získali textovou odpověď AI, musí se sestavit nový API požadavek, který se od toho předešlého liší pouze nepatrnými změnami.

Jedním z rozdílů je URL, která se v tomto případě z endpointu create změní na endpoint receive, před který se ještě vloží ID konverzace získané z předchozího requestu. V dalších částech requestu zůstává vše stejné.

```
url = "https://api.chatbotkit.com/v1/conversation/" + klic + "/receive"

data = {}

postfields = json.dumps(data)

headers = [
    'Authorization: Bearer sk-f5ccbfae299b07ed30c5b31cb7c82273060e33af78623b0abbd8de64f8a88f06',
    'accept: application/json',
    'Content-Type: application/json'
]
```

Obr. 7: Hlavičky a datová část druhého API requestu pro AI

Po zpracování a odeslání druhého requestu aplikace opět zpracuje data stejně jako v předchozím požadavku.

Z json odpovědi AI API (která nyní již obsahuje odpověď umělé inteligence) vytáhneme json položku „text", kterou dále porovnáváme. PhishGuard AI má ve svém backstory přesně nastaveno, v jakém formátu má odpovídat, a to „Legitimate: yes/no", tudíž stačí pouze porovnat, zda odpověď obsahuje řetězec „Legitimate: no" či nikoliv.

Pokud tato podmínka vyjde jako True, odečteme od integrity 50 % a zapíše se tato informace do databáze stejně jako u předchozí kontroly. Pokud je podmínka vyhodnocena jako False, znamená to, že AI vyhodnotila web jako legitimní, tudíž vše zůstane stejné, pouze se informace stejně jako v předchozím případě zapíše do globální PhishGuard databáze.

Nakonec se v modulu AI tvoří ještě jeden, poslední API požadavek, který pouze slouží pro vymazání konverzace, kterou jsme právě vytvořili, jelikož již není potřeba.

Tento požadavek se od ostatních dvou liší opět pouze endpointem v URL, který obsahuje ID následovaný endpointem delete. Tím se v platformě ChatBotKit smaže konverzace s daným ID. Tento přístup je ve PhishGuard vytvořen pouze z důvodu přehlednosti celého monitoringu AI, aby v budoucnu při větším použití nebyla platforma zahlcená zbytečně uchovávanými konverzacemi obsahujícími jednu jedinou zprávu.

```
url = "https://api.chatbotkit.com/v1/conversation/" + klic + "/delete"

data = {}

postfields = json.dumps(data)

headers = [
    'Authorization: Bearer sk-f5ccbfae299b07ed30c5b31cb7c82273060e33af78623b0abbd8de64f8a88f06',
    'accept: application/json',
    'Content-Type: application/json'
]
```

Obr. 8: Hlavičky a datová část třetího API requestu pro AI

4.3 Registrace domény

Další z obsažených kontrol je kontrola registrace domény. Tato kontrola je společně s phishing databází jedna z nejprimitivnějších, jejím účelem je zkontrolovat, jak dlouho je doména zaregistrovaná, jelikož jedním z dalších znaků typických phishing útoků je krátká životnost domén, na kterých jsou provozovány. K tomuto účelu se zde využívá knihovna whois.

Abychom mohli zjistit přesné datum vytvoření domény, musíme spustit metodu whois obsaženou ve stejnojmenné knihovně. Jediná potíž je v tom, že v naší aplikaci se URL zpracovává jako celková URL, ale servery whois očekávají pouze doménu.

Tohle se řeší jednoduchým způsobem pomocí funkce split, sloužící k rozdělování stringů podle určitých znaků. V našem případě split vypadá následovně: hostname.split(‘://’)[-1].split(‘/’)[0]. První část splitu dělí URL podle řetězce „://“, což znamená, že např. doménu https://soc.cz by rozdělil na části „https“ a „soc.cz“. Následující -1 v hranaté závorce udává, že bereme pouze poslední prvek, tudíž v ukázkovém příkladě pouze řetězec „soc.cz“. Druhá část splitu slouží pro domény s dalšími endpointy, například pokud by vstupem místo https://soc.cz byla URL adresa https://soc.cz/aktualni-rocnik, tato část splitu zajišťuje to, aby se řetězec rozdělil i podle posledního lomítka a následně se pokračovalo pouze s první částí, tudíž opět pouze s „soc.cz“.

Tímto přístupem je zajištěno, aby server whois dostal správný vstup, a už stačí jenom jednoduše vyslat dotaz na server pomocí metody whois a stejnojmenné funkce whois obsahující nově utvořený řetězec jako argument.

Po obdržení odpovědi od serveru whois aplikace kontroluje, zda existuje informace o datu vytvoření, a dále také stejným způsobem, jestli existuje informace o datu expirace domény. Obojí se kontroluje jednoduchou podmínkou if domain_info.creation_date nebo domain_info.expiration_date. Domain_info je proměnná, přes kterou je spouštěna funkce whois, tudíž prakticky vytvořena instance whois do dané proměnné. To znamená, že nadále proměnná obsahuje veškeré informace o doméně, tudíž pokud se tato podmínka vyhodnotí jako false, informace o datech dostupná není.

Pokud informace dostupná není, kontrola se vyhodnotí se stavem 2, to znamená, že neproběhla. Nijak to neovlivňuje integrity, pouze se do databáze zapíše informace, že tento způsob kontroly nelze provézt (je to ojedinělý případ, který může nastat pouze u této kontroly a kontroly SSL/TTL certifikátu).

Pokud ovšem obě data jsou nalezena, dále se zpracovávají a vyhodnocují, aby PhishGuard určil, zda je doména v tomto ohledu podezřelá nebo je v pořádku.

U data „CreationDate“ neboli data vytvoření se kontroluje, jak je doména stará. Pokud je zjištěno, že doména je mladší než 6 měsíců, je tato doména vyhodnocená jako podezřelá a z integrity se tudíž odebere 10 %. Následně se tato informace stejně jako u přechozích kontrol zapíše do globální databáze PhishGuard.

Podobná situace je u „“ExpirationDate“, kde se kontroluje, zda doména je stále platná, nebo je již tzv. „expirovaná“, to znamená, že majitel neprodloužil u hostingu její platnost. Pokud se jedná o druhý případ neboli o expirovanou doménu, je systémem taktéž vyhodnocena jako podezřelá a stejně jako u příliš nedávného data vytvoření v tomto případě systém odebere 10 % z integrity a opět informaci zapíše do databáze.

Kontrola registrace domény je sice ze všech nejméně technicky náročná, zato je ale extrémně účinná při rozhodování u webových stránek, které jsou na hraně toho, zda budou vyhodnoceny jako nebezpečné či pouze jako podezřelé.

4.4 SSL/TLS certifikát

Další z kontrol je kontrola SSL/TLS certifikátů. Tuto kontrolu již provádí většina moderních webových prohlížečů, avšak PhishGuard ji provádí mírně více do hloubky než samotné prohlížeče.

Tato kontrola je narozdíl od ostatních obsluhována v zdrojové kódu nikoliv pouze jednou metodou, ale metodami dvěma.

První z nich je je metoda trusted_ssl_ca(). Tato metoda tvoří seznam důvěryhodných vydavatelů certifikátů pro další použití v rámci SSL/TLS kontroly. Pomocí knihovny requests z oficiálního seznamu důvěryhodných ověřovatelů podle společnosti Mozilla získá danou tabulku vydavatelů, ze kterých pak vytáhne názvy daných ověřovatelů. Aby aplikace vytáhla pouze právě názvy z tabulky, která se nachází na již zmíněném webu, to je zařízeno pomocí knihovny bs4, která parsuje HTML staženéo pomocí requests a poté filtruje získaná data.

Nejprve nalezne veškeré řádky tabulky pomocí table.find_all('tr'), což aplikaci říká, aby našla všechny HTML tagy „tr", tedy řádky tabulek v HTML. Poté se přeskočí první řádek (jelikož se jedná o tzv. header řádek, neboli nadpis) a následně jsou pomocí table.find_all('td') nalezeny veškeré sloupce. Ty se následně ukládají do proměnné „columns". Název ověřovatele, který potřebujeme, se nachází pokaždé ve 3. sloupci, tudíž tuto informaci získáme pomocí columns[3].text.strip(): aplikace má vzít 3. sloupeček a vytáhnout z něj pouze obsažený text.

Tyto informace se následně uloží do pole „webovky" a takzvaně se normalizují: text se změní na pouze malá písmena a odstraní se veškeré mezery. Tento přístup je v aplikaci zvolen pro lepší porovnávání informací mezi naším seznamem a právě kontrolovaných certifikátem.

Druhou a hlavní metodou, při níž probíhá samotná kontrola, je check_ssl_certificate(). V první fázi je důležité, aby aplikace hlavně ověřila, zda se jedná o HTTPS webovou stránku či nikoliv neboli zda vůbec TLS certifikát existuje.

Tato fáze se zařizuje pomocí jednoduchého přístupu kontroly řetězců: PhishGuard zkontroluje, zda kontrolovaná URL začíná řetězcem https:// nebo ne. Pokud se zjistí že se jedná o případ kde web řetězcem https:// nezačíná , kontrola se v tu chvíli přeruší, od integrity se odebere 5 %, jelikož se jedná o nezabezpečenou webovou stránku, a tato informace se zapíše do globální databáze. Pokud PhishGuard v této fázi naopak zjistí, že tato podmínka neplatí, jiným slovy, webová stránka obsahuje TLS certifikát, kontrola dále pokračuje dalšími fázemi.

Další fáze kontroly již kontroluje samotný certifikát. Základem je utvoření TCP/IP připojení přes HTTPS s danou URL, aby bylo možné zajistit její certifikát. Tohle připojení se tvoří pomocí knihovny socket způsobem: socket.create_connection(hostname,443), což vytvoří potřebné připojení na daný hostname přes TCP na portu 443. Tento přistup je potřeba, jelikož HTTPS beží na TCP, tudíž před zahájením SSL/TLS handshaku se musí nejdříve otevřít připojení. Handshake se následně vytváří pomocí knihovny ssl, což je jedna z nativních knihoven obsažených v Pythonu ze základu. Tento handshake se pomocí context.wrap_socket(sock, server_hostname=hostname) naváže a následně vloží do proměnné sslsock, aby se s daným

připojením mohlo nadále pracovat. Pomocí této proměnné a následně funkce sslsock.getpeercert() aplikace získá TLS/SSL certifikát serveru, a následně z něj pak získává potřebné informace. Jde o název vydavatele/ověřovatele, common name (oficiální název domény) a datum expirace certifikátu.

Další a poslední fáze kontroly je ověření získaných informací z certifikátu. Nejprve se kontroluje datum expirace, které se jednoduše porovnává s dnešním datem; pokud je datum expirace dřívější než datum dnešní, jedná se o expirovaný certifikát. V tomto případě je v aplikaci odebráno 15 % z integrity, a jako obvykle se tato informace zanese do globální PhishGuard databáze. Jako další se kontroluje ověřovatel certifikátu. Pro tuto kontrolu je využita přechozí metoda, pomocí které se utvoří seznam důvěryhodných ověřovatelů a poté se porovnává ověřovatel uvedený na certifikátu právě s tímto seznamem. Pokud aplikace zjistí, že tento ověřovatel se v seznamu nenachází, jsou odebrána 3 % z integrity dané webové stránky a informace je zapsána do databáze.

Poslední je kontrola common name. Common name v certifikátu je informace o tom, pro jakou doménu certifikát platí. Jako příklad uvedeme certifikát webu https://soc.cz common name: pokud se jedná o legitimní web, common name bude soc.cz či *.soc.cz (wildcard CN, platí pro všechny weby končící soc.cz). V první řadě v kontrole CN PhishGuard zkontroluje, jestli se v certifikátu nenachází pravě zmíněné wildcard CN.Pokud ano, pro svoje interní účely z něj odstraní právě „*.“, aby se CN lépe porovnávalo s doménou. Poté se jednoduše porovná, zda CN je stejné jako kontrolovaná doména. Pokud ne, od integrity webu se odebere 10 % a tato informace se opět zapíše do globální databáze. Pokud z jakéhokoliv neočekávaného důvodu kontrola selže, vyhodnotí se jako nedokončená a od integrity se odeberou 3 %.

Veškeré zde zmíněné kontroly sice provádějí podobným způsobem samotné webové prohlížeče, avšak PhishGuard je obsahuje z toho důvodu, že se uživatel může stát i obětí MITM (Man in the middle) útoku či jiného způsobu podvrženého SSL/TLS certifikátu, což by samotný prohlížeč nebyl schopný odhalit. Narozdíl od prohlížeče dokáže PhishGuard v této kontrole odhalit i takto extrémní případy.

4.5 Redirects (přesměrování)

Známým znakem podvodných webů je několikanásobné přesměrování na jiné domény před tou finální ve snaze zabránit odhalení. Také na kontrolu tohoto znaku se PhishGuard zaměřuje.

Stejně jako k předchozí úkolům síťového rázu se zde využívá knihovna requests. Nejdřív se obsah webu stáhne pomocí requests.get(url, allow_redirects = true), což stáhne webovou stránku a následně sleduje veškeré přesměrování. Aby bylo možné odchytávat pouze podezřelé přesměrování (pokud se jedná o přesměrování v rámci stejné domény, není považováno za podezřelé), musíme získat doménu původní URL adresy, a to pomocí urlparse(url).netlock.

Ze všech provedených přesměrování se utvoří pole, které se následně cyklicky prochází. V daném cyklu se kontroluje, zda se doména přesměrování shoduje s původní doménou; pokud ne, toto přesměrování se přidá do dalšího pole. Následně se ještě zkontroluje finální doména; pokud se liší, vkládá se taktéž do pole podezřelých přesměrování.

V poslední řadě PhishGuard zkontroluje, zda pole podezřelých přesměrování je prázdné. Pokud ne, jedná se o podezřelé chování, od integrity se odebere 25 %, a opět se daná informace zapíše do globální databáze.

4.6 Formuláře

Poslední z kontrol je kontrola „POST FORMS“, neboli kontrola formulářů obsažených na dané webové stránce.

V tomto případě existují 2 typy podezřelého chování, a to formuláře, které odkazují nikam, a ty, které odkazují na jinou doménu, než je doména webu. PhishGuard kontroluje oba případy.

Základem je získat všechny formuláře, tudíž se nejprve pomocí knihovny bs4 a metody BeautifulSoup() stáhne veškerý obsah HTML kontrolovaného webu. Následně se vyhledávají veškeré formuláře pomocí soup.find_all(), a všechny se vloží do pole „forms“, aby bylo možné nadále s nimi pracovat.

Pomocí cyklu se všechny formuláře obsažené na webu procházejí a kontroluje se, kam odkazují. Toto se vykonává podle metody forms.get(‘action’), která získá hodnotu atributu action neboli URL, na kterou se formulář odesílá. Pokud je action tzv. relativní (např. /submit.php), pomocí urljoin() se převede na plnou URL adresu. Tímto způsobem se přesně získá URL, na kterou formulář vede, tato informace se pak následně dál kontroluje.

Pokud se v předchozím kroku žádná URL nezískala, znamená to, že formulář nikam nevede. Jde tedy o druhý případ podezřelého formuláře, od integrity se odečte 25 % a informace se zanese do globální databáze.

Z URL, kterou jsme získali z formuláře, musíme získat finální adresu. Stejně jako v REDIRECTS kontrole se URL přes requests kontaktuje včetně povolení přesměrování, a z toho se následně získává finální URL adresa. Ta se porovnává s originální URL adresou, a tím se kontroluje, zda se domény shodují, nebo jestli se liší. Pokud se liší, aplikace odebere od integrity webové stránky 25 % a jako obvykle tuto informaci zapíše do globální databáze.

5 PHISHGUARD JAKO CELEK

Jak již bylo zmíněno v kapitole 3.3, veškeré dílčí části jsou zaobaleny do metody „new_url()“, která veškeré funkce a kontroly zaobaluje a tvoří tak celou finální aplikaci PhishGuard. Metoda new_url() zpracuje příchozí URL adresu od rozšíření do prohlížeče.

U dané URL nejdříve zkontroluje, zda se již nachází v globální databázi. Pokud ano, aplikace-uživateli zobrazí již dříve získané informace o dané webové stránce. Pokud ne, danou URL předává postupně všem kontrolám.

Kontroly jsou postupně ve stejném pořadí spouštěny přes tzv. thready neboli vlákna, aby bylo možné v případě potřeby kontroly přerušovat.

Na závěr PhishGuard vyhodnotí, v jakém rozmezí se nachází hodnota integrity, a podle toho zobrazí správnou vizuální informaci, zda je webová stránka nebezpečná, podezřelá či bezpečná. Výsledkem tohoto technologického řešení je moderní a funkční ochrana proti phishingu.

6 ZÁVĚR

V úvodu vývoje aplikace PhishGuard jsem si stanovil jediný cíl, a to analyzovat mechanismy a způsoby, které používají podvodníci ke klamání obětí pomocí metod phishingu, a na základě těchto poznatků vytvořit moderní, intuitivní a účinnou ochranu, která pokryje nedostatky dosud existujících řešení.

Tento cíl jsem splnil vytvořením systému, který kombinuje několik detekčních metod a vyhodnocuje tak přesně a téměř bezchybně úroveň důvěryhodnosti všech webových stránek, na kterých se uživatel ocitne.

Do budoucna je možnétuto aplikaci mnoda dalšími způsoby efektivně vylepšovat a vyrovnat se tak technologickému pokroku podvodníků.

Celkově lze konstatovat, že systém PhishGuard poskytuje efektivní řešení na obranu proti prudce rostoucím podvodům na internetu. Jeho další vývoj a podpora by mohly přispět k vyšší přesnosti, adaptabilitě na nové hrozby a celkově většímu pokroku v oblasti kyberbetické bezpečnosti.

7 POUŽITÁ LITERATURA

FLASK. Flask Documentation [online]. Pallets, 2023 [cit. 2025-02-11]. Dostupné z: https://flask.palletsprojects.com

PYTHON. Python Documentation [online]. Python Software Foundation, 2023 [cit. 2025-02-11]. Dostupné z: https://docs.python.org

GOOGLE SAFE BROWSING. Google Safe Browsing API [online]. Google, 2023 [cit. 2025-02-11]. Dostupné z: https://developers.google.com/safe-browsing

CHATBOTKIT. Chatbotkit API Documentation [online]. Chatbotkit, 2023 [cit. 2025-02-11]. Dostupné z: https://chatbotkit.com/docs

WEBEXTENSIONS. Chrome Extension Developer Guide [online]. Google, 2023 [cit. 2025-02-11]. Dostupné z: https://developer.chrome.com/docs/extensions/

8 SEZNAM OBRÁZKŮ A TABULEK

Obr. 1: Prvotní uživatelské rozhraní....9
Obr. 2: Minimalizované notifikační okno....10
Obr. 3: Expandované notifikační okno....10
Obr. 4: Background script pro rozšíření do prohlížeče....11
Obr. 5: Hlavičky a payload API requestu pro GSB....15
Obr. 6: Hlavičky a datová část prvního API requestu pro AI....17
Obr. 7: Hlavičky a datová část druhého API requestu pro AI....18
Obr. 8: Hlavičky a datová část třetího API requestu pro AI....19